

```
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 51.828 s
[INFO] Finished at: 2023-12-05T23:16:46+05:30
[INFO] -----
```

Let's now see the structure of the project, but before that install the *tree* command in Ubuntu.

```
test@ubuntu:~/Desktop$ sudo apt-get install tree
```

Next, use the *tree* command with the directory name:

```
test@ubuntu:~/Desktop$ tree my-app
```

```
my-app
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── osfy
│   │   │   │   │   ├── app
│   │   │   │   │   │   App.java
│   │   └── test
│   │       ├── java
│   │       │   ├── com
│   │       │   │   ├── osfy
│   │       │   │   │   ├── app
│   │       │   │   │   │   AppTest.java
```

11 directories, 3 files

In the next section, we will execute unit tests and create packages.

Executing unit tests and creating a package using Jenkins

Go to the application directory created earlier and execute the 'mvn test' command:

```
test@ubuntu:~/Desktop$ cd my-app/
test@ubuntu:~/Desktop/my-app$ mvn test

[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.osfy.app:my-app >-----
[INFO] Building my-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
```

```
Downloading from central: https://repo.maven.apache.org/
maven2/org/apache/maven/plugins/maven-resources-plugin/3.0.2/
```

```
maven-resources-plugin-3.0.2.pom
.
.
.
[INFO] T E S T S
[INFO] -----
--
[INFO] Running com.osfy.app.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time
elapsed: 0.01 s - in com.osfy.app.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.542 s
[INFO] Finished at: 2023-12-05T23:19:34+05:30
[INFO] -----
```

Now, let's create a JAR file using the 'mvn package' command:

```
test@ubuntu:~/Desktop/my-app$ mvn package
```

Verify the target folder inside the application directory and you will find a newly created JAR file.

Let's execute the JAR file:

```
test@ubuntu:~/Desktop/my-app$ java -cp target/my-app-1.0-
SNAPSHOT.jar com.osfy.app.App
```

Hello World!

So now we have a simple Java application with unit tests. Let's try to achieve the same steps using Jenkins. How is this useful in DevOps implementation?

Jenkins is an open source tool that provides integration with the existing tools used in application life cycle management to automate all activities — build, test, and deploy. It is almost a synonym for DevOps and is easy to learn. It has a huge community for support. Jenkins provides a simple way to create a pipeline that includes continuous integration and continuous delivery. It can be used creatively for managing entire life cycle management activities easily. Plugin-based architecture helps to use the existing tools in the pipeline and, hence, too many changes in culture in one go can be avoided.

Download Jenkins from <https://www.jenkins.io/download/>